



Практичні поради Python

Практичні поради Python

Як і в будь-якій іншій мові чи інструменті, Python має кілька найкращих практик, яких слід дотримуватися до, під час і після процесу написання коду. Вони роблять код читабельним і створюють стандарт для всієї галузі. Інші розробники, які працюють над проектом, повинні мати можливість читати та розуміти ваш код. Ми перерахували кілька з них, щоб ви дотримувались і писали чистіший і професійніший код. Ви дотримуетесь чогось із цього?

Отже, ось кілька важливих найкращих практик Python, які ви завжди повинні пам'ятати.

1. Створіть репозиторій коду та запровадьте контроль версій

Якщо ви коли-небудь були на GitHub, ви, мабуть, помітили, що структура звичайного проекту виглядає так:

```
docs/conf.py
docs/index.rst
module/__init__.py
module/core.py
tests/core.py
LICENSE
README.rst
requirements.txt
setup.py
```



*re***Structured***Text*

Структура проекту Python

Коли ви починаєте новий проект python, ви можете створити сховище коду та реалізувати контроль версій. GitHub є одним із таких постачальників, але ви можете використовувати будь-який інший. Давайте поговоримо про структуру проекту, але перед цим я рекомендую вам перевірити останній проект DataFlair на Python щодо розпізнавання мовних емоцій.

Ліцензія

Це в кореновому каталозі, куди ви повинні додати ліцензію для вашого проекту. Деякі ліцензії:

GNU AGPLv3

GNU GPLv3

GNU LGPLv3

Mozilla Public License 2.0

Apache License 2.0

MIT License

The Unlicense

Структура проекту Python

README

Це також у кореневому каталозі, де ви описуєте свій проект і що він робить. Ви можете написати це у Markdown, reStructuredText або звичайний текст.

Module Code

Тут зберігається ваш фактичний код, який може бути в підкаталозі або в кореневому каталозі.

requirements.txt

Це не є обов'язковим, але якщо ви використовуєте це, ви розміщуєте його в кореневому каталозі. Тут ви згадуєте модулі та залежності проекту – речі, без яких він не працюватиме.

Структура проекту Python

setup.py

Цей скрипт у корені дозволяє Distutils створювати та розповсюджувати модулі, необхідні для проекту.

Documentation

Читальна документація є важливою. Це розміщено в каталозі документів.

Tests

Більшість проектів мають тести – зберігайте їх у каталозі tests.

2. Створюйте читабельну документацію

Отже, наступним у найкращих практиках python є доступна для читання документація. Як ми щойно сказали, читабельна документація важлива. Ви можете вважати це обтяжливим, але воно створює чистий код. Для цього можна використовувати Markdown, reStructuredText, Sphinx або docstrings. Markdown і reStructuredText — це мови розмітки із синтаксисом форматування простого тексту, який полегшує розмітку тексту та перетворення його у такий формат, як HTML або PDF. reStructuredText дозволяє створювати вбудовану документацію. А Sphinx — це інструмент для легкого створення розумної та красивої документації. Це дозволяє генерувати документацію Python з існуючого reStructuredText. Він також дозволяє експортувати документацію у такі формати, як HTML. Рядки документації — це рядки документації на початку кожного модуля, класу або методу.

3. Дотримуйтесь правил стилю

PEP8 містить кілька чудових пропозицій, створених спільнотою. PEP розшифровується як Python Enhancement Proposals – це рекомендації та стандарти для розробки. Існують інші PEP, такі як PEP20, але PEP8 — це біблія спільноти Python для вас, якщо ви хочете правильно стилізувати свій код. Це зроблено для того, щоб переконатися, що весь код Python виглядає та відчувається однаково. Однією з таких інструкцій є іменування класів за угодою CapWords.

- Використовуйте правильні правила іменування змінних, функцій, методів тощо.
- Змінні, функції, методи, пакети, модулі: `this_is_a_variable`
- Класи та винятки: CapWords
- Захищені методи та внутрішні функції: `_single_leading_underscore`
- Приватні методи: `__double_leading_underscore`
- Константи: `CAPS_WITH_UNDERSCORES`

Використовуйте 4 пробіли для відступу. Додаткові умовні позначення див. у PEP8.

4. Негайно виправляйте зламаний код

Як і у випадку з теорією зламаного коду, негайно виправте свій зламаний код. Якщо ви залишите це, поки ви працюєте над чимось іншим, це може призвести до гірших проблем пізніше. Це те, що робить Microsoft. Колись він мав жахливий цикл виробництва з першою версією MS Word. Тож тепер він дотримується «методології нульових дефектів» і завжди виправляє помилки та дефекти, перш ніж продовжити.

5. Використовуйте PyPI замість того, щоб робити це самостійно

Однією з причин популярності Python є PyPI — це індекс пакетів Python; на момент написання статті нараховується понад 198 190 проектів. Це сховище програмного забезпечення для Python і містить тисячі проектів Python. Ви повинні використовувати цей код замість того, щоб писати його самостійно - це економить час і дозволяє зосередитися на більш важливих речах. У PyPI є проекти про все, і ви можете встановити їх за допомогою `pip`. Ви також можете створити та завантажити свій власний пакет тут. Дотримуйтеся цієї найкращої практики в Python, це вам дуже допоможе.

6. Дзен в Python

Тім Пітерс написав цей короткий вірш, щоб висловити, яких цінностей слід дотримуватися під час кодування на Python. Ви можете отримати це, запустивши

```
«імпортувати це» в IDLE:  
1. >>> import this
```

The Zen of Python, by Tim Peters

Красиве краще, ніж потворне.
Явне краще, ніж неявне.
Просте краще, ніж складне.
Складне краще, ніж складне.
Плоский краще, ніж вкладений.
Рідкий краще, ніж щільний.
Читабельність має значення.
Особливі випадки не настільки особливі,
щоб порушувати правила.
Хоча практичність перемагає чистоту.
Помилки ніколи не повинні проходити
мовчки.
Якщо явно не замовчувати.
Зіткнувшись з двозначністю, відмовтеся
від спокуси вгадати.
Має бути один — і бажано тільки один —
очевидний спосіб зробити це.

7. Використовуйте правильні структури даних

Дізнайтеся про переваги та обмеження різних структур даних і виберіть правильну для свого коду.

8. Напишіть читабельний код

Ви повинні використовувати розриви рядків і робити відступи в коді. Використовуйте угоди про найменування ідентифікаторів – це полегшить розуміння коду. Використовуйте коментарі та пробіли навколо операторів і призначень. Зберігайте максимальну довжину рядка 79 символів. Ви повинні залишатися послідовними та писати читабельний код.

9. Використовуйте віртуальні середовища

Вам слід створити віртуальне середовище для кожного проекту, який ви створюєте. Це дозволить уникнути конфліктів бібліотек, оскільки для різних проектів можуть знадобитися різні версії певної бібліотеки.

10. Напишіть об'єктно-орієнтований код

Python є об'єктно-орієнтованою мовою, і все в Python є об'єктом. Ви повинні використовувати об'єктно-орієнтовану парадигму, якщо пишете код для Python. Це має переваги приховування даних і модульності. Це дозволяє багаторазове використання, модульність, поліморфізм, інкапсуляцію даних і успадкування. Крім того, ви повинні писати модульний і неповторюваний код. Використовуйте класи та функції у своєму коді.

11. Чого не робити

Уникайте імпорту всього з пакета - це забруднює глобальний простір імен і може спричинити зіткнення. Не використовуйте кращі практики з інших мов. Не вимикайте звіти про помилки під час розробки – вимкніть їх після. Не змінюйте `sys.path`, використовуйте для цього `distutils`.



HTTP Requests

HTTP (HyperText Transfer Protocol)

HTTP — це **основний протокол вебу**, розроблений для передачі гіпертекстових документів, таких як HTML. Він є фундаментом роботи Інтернету: дозволяє браузерам (клієнтам) запитувати вебсторінки із серверів.

Через різні методи запитів HTTP забезпечує можливість виконувати дії, як-от:

- отримання (retrieving),
- відправлення (sending),
- оновлення (updating),
- або видалення (deleting) даних на сервері.

URL (Uniform Resource Locator)

URL — це **універсальний спосіб адресації ресурсів в Інтернеті**: вебсторінок, зображень, файлів тощо.

Кожен URL містить:

- **протокол** (наприклад, HTTP або HTTPS),
- **доменне ім'я** сервера, де розміщено ресурс,
- **шлях (path)**, який вказує на розташування ресурсу на сервері.

Це дозволяє браузеру точно знайти і завантажити потрібний контент з Інтернету.

Запити та відповіді

Взаємодія між клієнтом і сервером в Інтернеті базується на **обміні запитами й відповідями**.

Коли користувач запитує вебсторінку:

1. Його браузер надсилає **HTTP-запит** до сервера.
2. Сервер **обробляє запит**.
3. У відповідь сервер надсилає **HTTP-відповідь**, яка може містити сторінку, дані або повідомлення про помилку.

Цей процес лежить в основі роботи мережі, забезпечуючи **отримання, надсилання та обробку** вебконтенту.

Методи запитів HTTP

HTTP визначає набір **методів запитів**, кожен з яких відповідає за певну дію над ресурсами:

- **GET** — отримати ресурс без змін;
- **POST** — надіслати дані на сервер (наприклад, при заповненні форми);
- **PUT** — оновити існуючий ресурс;
- **PATCH** — часткове оновлення ресурсу;
- **DELETE** — видалити ресурс.

Ці методи забезпечують структурований підхід до керування вебданими.

Коди стану

Коди стану HTTP — це **показники результату запиту** до сервера.

Вони поділяються на кілька категорій:

Категорія	Значення
1xx	Інформаційні (Informational)
2xx	Успішні (Success)
3xx	Перенаправлення (Redirection)
4xx	Помилка клієнта (Client Error)
5xx	Помилка сервера (Server Error)

- ♦ **200 OK** — запит виконано успішно.
- ♦ **404 Not Found** — запитуваний ресурс не знайдено.

Коди стану допомагають діагностувати проблеми й розуміти, як сервер обробив запит.

Заголовки та тіло запиту

У протоколі HTTP **заголовки (headers)** і **тіло (body)** мають різні ролі:

- **Headers** містять **метадані** — тип контенту, довжину, cookies тощо.
- **Body** містить **основні дані**, які передаються (наприклад, текст форми чи JSON).

Розуміння структури та призначення заголовків і тіла запиту важливе для ефективної взаємодії з вебсервісами й API.

REST (Representational State Transfer)

REST — це **архітектурний стиль** розробки мережевих застосунків, що базується на **stateless-комунікації** (без збереження стану між запитами) і використовує **HTTP** як основний протокол.

REST орієнтований на **ресурси**, а не на дії:

- кожен ресурс має унікальний **URL**;
- стандартні методи HTTP (GET, POST, PUT, DELETE) використовуються для взаємодії з ресурсами.

RESTful-підхід забезпечує **простоту, масштабованість і легкість інтеграції**, тому широко використовується для створення **API**.

Data Formats: JSON and XML (Формати даних: JSON і XML)

У вебсервісах і при роботі з API дані найчастіше передаються у форматах:

- **JSON (JavaScript Object Notation)**
 - **XML (eXtensible Markup Language)**
-
- ♦ **JSON** — легкий, зручний для читання, ідеальний для сучасних веб-API.
 - ♦ **XML** — більш структурований і докладний, підходить для складних обмінів даними.

Обидва формати дозволяють стандартизовано й ефективно обмінюватися структурованими даними.

Бібліотеки для HTTP-запитів у Python

Python має кілька бібліотек, які роблять роботу з HTTP-запитами **простішою та ефективнішою**.

 Основні з них:

- **requests** — найпопулярніша бібліотека з інтуїтивним інтерфейсом. Дозволяє легко надсилати HTTP-запити та обробляти відповіді.
- **http.client** — стандартна бібліотека для **низькорівневої роботи** з HTTP і HTTPS-з'єднаннями.
- **urllib** — більш складна, але потужна бібліотека для роботи з URL, включно зі створенням HTTP-запитів.

Вибір бібліотеки залежить від потреб вашого застосунку — простота, гнучкість або повний контроль.

Сесійні об'єкти в Python

Сесійні об'єкти, особливо в бібліотеці **requests**, дозволяють **зберігати стан між кількома HTTP-запитами**.

- ♦ За допомогою сесій можна повторно використовувати параметри, такі як:
 - cookies,
 - заголовки (headers),
 - параметри авторизації.

Це підвищує ефективність запитів і зменшує затрати часу на повторне встановлення з'єднання.

Сесії особливо корисні для застосунків, які взаємодіють із вебсервісами, що потребують **авторизації або постійного стану**.

Cookies у протоколі HTTP

Cookies — це ключовий елемент для **збереження стану** у вебзастосунках.

Вони дозволяють серверам надсилати інформацію про стан клієнту, який потім передає її назад із наступними запитами.

Цей механізм потрібен для:

- користувацьких сесій (логін/авторизація),
- персоналізованого контенту,
- аналітики та відстеження активності користувачів.

Розуміння принципів роботи з cookies важливе як для **розробників бекенду**, так і для **клієнтських застосунків**, щоб забезпечити **безпечну та стабільну роботу**.

Безпека при роботі з HTTP-запитами

Безпека — критично важливий аспект при роботі з HTTP-запитами, особливо під час обробки **конфіденційних даних**.

Рекомендації:

- Завжди використовуйте **HTTPS** для шифрованого з'єднання.
- Перевіряйте **SSL-сертифікати**.
- Обережно працюйте з cookies та сесійними даними.
- Валідуйте всі отримані ззовні дані перед обробкою.

Такі практики допомагають уникнути:

- атак типу **injection**,
- витоку даних,
- несанкціонованого доступу.

Дотримання принципів безпеки гарантує **захист даних** і довіру користувачів до вашого застосунку.



Library Requests

Вступ до бібліотеки `requests`

Бібліотека **requests** у Python — це **зручний і потужний HTTP-клієнт** для відправлення та отримання HTTP-запитів і відповідей.

На відміну від стандартних бібліотек (наприклад, **urllib**), **requests** надає **простіший, зрозуміліший інтерфейс** для взаємодії з вебресурсами.

♦ Основні можливості:

- автоматичне керування сесіями,
- підтримка cookies,
- просте надсилання GET/POST-запитів,
- робота з JSON,
- обробка заголовків, файлів і параметрів.

Саме тому **requests** — це стандартна бібліотека, яку найчастіше використовують для роботи з RESTful API та вебсервісами.

(Основні можливості бібліотеки requests)

Бібліотека `requests` поєднує **потужність** і **простоту використання**.

Ключові функції:

- Легке створення HTTP-запитів **GET** та **POST**.
- Автоматичне **URL-кодування**.
- Підтримка **формових даних (form data)**, завантаження файлів та налаштування заголовків.
- **Перевірка SSL-сертифікатів** за замовчуванням.
- Підтримка **cookies** і **сесій**.

Ці можливості охоплюють більшість сценаріїв роботи з вебзапитами у сучасних застосунках Python.

Перші кроки з requests

Почати роботу з `requests` дуже просто.

Якщо бібліотека ще не встановлена, додайте її у середовище Python командою:

```
pip install requests
```

Ця єдина команда встановлює `requests` та готує середовище до повноцінної роботи з HTTP-запитами.

Виконання GET-запитів

У `requests` це робиться дуже просто:

```
import requests
```

```
response = requests.get('https://example.com/api/data')
```

Цей рядок надсилає GET-запит на вказаний URL і зберігає відповідь сервера у змінній `response`.

Щоб отримати вміст відповіді:

```
response.text
```

для звичайного тексту, або

```
response.json()
```

для JSON-даних.

```
import requests

response = requests.get('https://example.com/api/data')

print(response.text)

json_response = response.json()
print(json_response)
```

Обробка відповідей

Кожен HTTP-запит у `requests` повертає **об'єкт відповіді**, який містить усю інформацію від сервера.

Основні атрибути:

`response.status_code` # код стану HTTP

`response.headers` # заголовки відповіді

`response.text` # тіло відповіді (як текст)

`response.json()` # автоматичне розпарсення JSON

Уміння правильно обробляти відповіді дозволяє ефективно взаємодіяти з вебданими та діагностувати можливі помилки.

```
import requests

response = requests.get('https://example.com/api/data')

if response.status_code == 200:
    print("Request successful!")
else:
    print("Request failed with status code:", response.status_code)

print("Response headers:", response.headers)
print("Response content:", response.text)

try:
    json_response = response.json()
    print("JSON response:", json_response)
except ValueError:
    print("Response does not contain valid JSON.")
```

Відправлення POST-запитів

POST-запити використовуються для **надсилання даних на сервер** — наприклад, під час авторизації або надсилання форм.

У `requests` це робиться так:

`requests.post(url, data={'key': 'value'})`

АБО

`requests.post(url, json={'key': 'value'})`

— надсилає **JSON POST-запит**.

Така гнучкість дозволяє зручно взаємодіяти з API, які приймають дані у різних форматах.

```
import requests

url = 'https://api.example.com/submit'

form_data = {'key': 'value'}

response = requests.post(url, data=form_data)

if response.status_code == 200:
    print("Успішно надіслано дані!")
    print("Відповідь сервера:", response.text)
else:
    print("Помилка при відправці даних:", response.status_code)
```

Автентифікація з requests

Багато вебсервісів потребують **автентифікації** перед обміном даними.

Бібліотека `requests` робить цей процес простим і зрозумілим.

Для базової автентифікації (Basic Auth): `requests.get(url, auth=('user', 'pass'))`

Бібліотека автоматично додає заголовок авторизації до запиту.

Для складніших випадків (наприклад, **OAuth** або **Token-based authentication**) `requests` також має відповідну підтримку.

Таким чином, можна легко налагодити **безпечну взаємодію з вебсервісами**.

```
import requests

url = 'https://api.example.com/data'

auth_params = ('username', 'password')

response = requests.get(url, auth=auth_params)

if response.status_code == 200:
    print("Дані отримано успішно!")
    print("Вміст відповіді:", response.text)
else:
    print("Помилка при отриманні даних:", response.status_code)
```

Робота з cookies

Cookies важливі для **підтримання сесійного стану** у взаємодії з вебсервісами.

Бібліотека `requests` дозволяє дуже просто працювати з ними.

Відправлення cookies: `requests.get(url, cookies={'session_id': '12345'})`

Отримання cookies від сервера: `response.cookies`

Цей функціонал зручний для **автоматизації дій**, які вимагають входу користувача або збереження стану між запитами.

```
import requests

response = requests.get('https://api.example.com/data', cookies={'session_id': '123456'})

if response.status_code == 200:
    print("Запит успішно виконано!")
    print("Вміст відповіді:", response.text)
else:
    print("Помилка при виконанні запиту:", response.status_code)

received_cookies = response.cookies
print("Отримані cookies:", received_cookies)
```

Сесії у бібліотеці requests

Для тривалої взаємодії з вебсервісом варто використовувати **сесії**.

Це дозволяє повторно використовувати cookies і заголовки між кількома запитами.

with requests.Session() as session:

 session.get(url)

 session.post(url, data=data)

 session.post(url, data=data)

Сесії оптимізують роботу мережевих запитів і зменшують затримки, особливо якщо потрібно виконати кілька послідовних звернень до API.

```
import requests
```

```
with requests.Session() as session:
```

```
    response1 = session.get('https://example.com/login')
```

```
    response2 = session.post('https://example.com/login', data={'username': 'user', 'password': 'pass'})
```

```
    response3 = session.get('https://example.com/dashboard')
```

```
    response4 = session.get('https://example.com/profile')
```

Обробка помилок і винятків

Надійна обробка помилок — основа стабільного застосунку.

Бібліотека `requests` має власні **класи винятків**, які допомагають керувати помилками мережевих запитів.

```
import requests

from requests.exceptions import RequestException, HTTPError

try:
    response = requests.get(url)
    response.raise_for_status()
except HTTPError as e:
    print("HTTP error:", e)
except RequestException as e:
    print("Network error:", e)
```

Цей підхід дозволяє безпечно обробляти збої, тайм-аути або неправильні відповіді від сервера.

```
import requests

try:
    response = requests.get('https://example.com/api')
    response.raise_for_status()

except requests.exceptions.HTTPError as err:
    print(f"HTTP error occurred: {err}")

except requests.exceptions.ConnectionError as err:
    print(f"Error connecting to the server: {err}")

except requests.exceptions.Timeout as err:
    print(f"Timeout error: {err}")

except requests.exceptions.RequestException as err:
    print(f"An unexpected error occurred: {err}")
```

Практичні приклади з requests

Бібліотека `requests` має **широкий спектр застосувань**, від простих до складних сценаріїв:

- ◆ Отримання даних з API для аналітики.
- ◆ Вебскрапінг (витяг даних зі сторінок).
- ◆ Автоматизація взаємодії з вебформами.
- ◆ Тестування RESTful API.

`requests` — це **універсальний інструмент**, який підходить для всіх рівнів розробників, від початківців до професіоналів.

Висновки та ресурси

Бібліотека **requests** — незамінний інструмент для Python-розробників, які працюють із вебзапитами. Вона поєднує **простоту, стабільність і потужність**, охоплюючи майже всі типові завдання при роботі з HTTP.

 Для подальшого вивчення рекомендується:

- офіційна документація: <https://requests.readthedocs.io/en/latest/>
- спільноти розробників і форуми
- власна практика: експериментуйте, створюйте й аналізуйте HTTP-запити

Питання та відповіді